

Git for Developers

Module 9 — Git Internals

Object Model, `.git` Folder, Hashing, Hooks, and Plumbing

Module 9 — Agenda

Under the Hood

- How Git stores data: objects, blobs, trees, and commits
- The `.git` folder structure explained
- SHA-1 / SHA-256 object hashing

Automation and Low-Level Access

- Git hooks: client-side and server-side automation
- Plumbing vs. porcelain commands

Part 1 — How Git Stores Data

Git's Content-Addressable Object Store

Git stores everything as **objects** in `.git/objects/`. Each object is:

- Identified by its **SHA-1 hash** (40 hex characters)
- **Immutable** — changing content produces a new hash
- **Deduplicated** — identical content = identical hash = stored once

The four object types

Type	Contains	Analogous to
blob	File content	A file
tree	Directory listing (blobs + trees + names)	A folder
commit	Tree pointer + parent + metadata	A snapshot
tag	Annotated tag metadata	A label

Blob — File Content

A blob stores the raw content of a file (no filename, no path):

```
# Hash and store a file as a blob manually
echo "Hello, Git!" | git hash-object --stdin -w
# Output: 8ab686...

# Read a blob by its hash
git cat-file -p 8ab686eafeb1f44702738c8b0f24f2567c36da6d
# Hello, Git!

# Check object type
git cat-file -t 8ab686eafeb1f44702738c8b0f24f2567c36da6d
# blob
```

Tree — Directory Snapshot

A tree maps names to blobs and other trees:

```
# Inspect a tree object  
git cat-file -p HEAD^{tree}
```

```
100644 blob a3f7c21... README.md  
100644 blob b2e1d40... package.json  
040000 tree c9a8b12... src  
040000 tree d7f3e56... tests
```


Field	Meaning
100644	Mode (regular file)
040000	Mode (directory / sub-tree)
blob / tree	Object type
SHA hash	Content identifier
Name	File or directory name

Commit — Snapshot with Context

```
git cat-file -p HEAD
```

```
tree    c9a8b12a3f7c218b9e4c1d2f3a4b5c6d7e8f9a0b
parent  b2e1d40f3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8
author  Jane Smith <jane@example.com> 1710760980 +0100
committer Jane Smith <jane@example.com> 1710760980 +0100

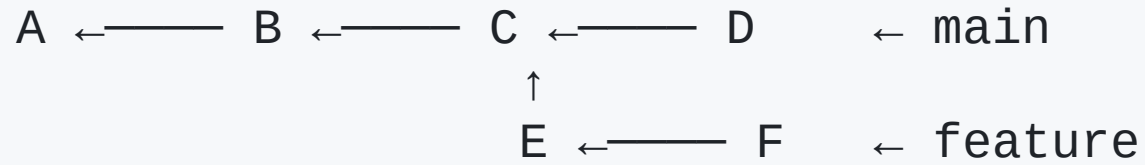
feat: add JWT refresh token rotation
```

What the commit contains

- **tree** — SHA of the root tree (the entire project at this moment)
- **parent** — SHA of the previous commit (forms the chain)
- **author / committer** — identity and timestamp
- **message** — the commit message

The Directed Acyclic Graph

Commits form a **DAG** (Directed Acyclic Graph):



- Each arrow = "parent of" relationship
- Branches are just pointers to a specific commit
- **HEAD** points to the current branch pointer
- The graph is **immutable** — commits never change SHA

Part 2 — The `.git` Folder

`.git/` Structure

<code>.git/</code>	
— <code>HEAD</code>	← points to current branch (e.g., <code>ref: refs/heads/main</code>)
— <code>config</code>	← repository-local git config
— <code>description</code>	← used by GitWeb (mostly irrelevant today)
— <code>index</code>	← the staging area (binary format)
— <code>COMMIT_EDITMSG</code>	← message from the last commit
— <code>objects/</code>	← all Git objects (blobs, trees, commits, tags)
— <code>pack/</code>	← packed objects for efficiency
— <code>info/</code>	
— <code>refs/</code>	
— <code>heads/</code>	← local branch pointers (one file per branch)
— <code>tags/</code>	← tag pointers
— <code>remotes/</code>	← remote-tracking branch pointers
— <code>origin/</code>	
— <code>hooks/</code>	← executable scripts for Git events

Key Files in `.git/`

```
# HEAD – what branch you're on
$ cat .git/HEAD
ref: refs/heads/main

# Branch pointer – the commit that branch points to
$ cat .git/refs/heads/main
a3f7c218b9e4c1d2f3a4b5c6d7e8f9a0b1c2d3e4

# How objects are stored (first 2 chars = directory)
$ ls .git/objects/a3/
f7c218b9e4c1d2f3a4b5c6d7e8f9a0b1c2d3e4
```

Part 3 — Object Hashing

SHA-1 and the Move to SHA-256

Git historically uses **SHA-1** to identify objects:

```
# Git computes: SHA1("blob " + content_length + "\0" + content)
$ echo -n "Hello, Git!" | git hash-object --stdin
8ab686eafeb1f44702738c8b0f24f2567c36da6d
```

The SHA-256 transition

- SHA-1 is cryptographically weak (SHAttered attack, 2017)
- Git 2.29+ supports SHA-256 repositories (`git init --object-format=sha256`)
- SHA-256 hashes are 64 hex characters vs. 40 for SHA-1
- The transition is gradual — most repositories still use SHA-1

Content Addressing in Practice

```
# Two files with identical content produce the same blob hash
echo "Hello" > a.txt
echo "Hello" > b.txt
git hash-object a.txt
git hash-object b.txt
# Same hash! Git stores it only once.

# Verify the integrity of the entire repository
git fsck
# Traverses every object, checks all hashes
```

Git's SHA-based storage is what makes it trustworthy — any corruption changes the hash and is immediately detectable.

Part 4 — Git Hooks

What Are Git Hooks?

Git hooks are **executable scripts** in `.git/hooks/` that run automatically at specific points in the Git workflow.

```
.git/hooks/  
├── pre-commit      ← runs before a commit is created  
├── commit-msg      ← runs after commit message is entered  
├── post-commit     ← runs after a commit is created  
├── pre-push        ← runs before git push  
├── pre-rebase      ← runs before a rebase  
├── post-checkout    ← runs after git checkout  
└── update          ← server-side: runs before a branch is updated
```

Sample hooks (`.sample` extension) are included — remove `.sample` and make executable to activate.

Client-Side Hooks

Hook	When it runs	Common use
<code>pre-commit</code>	Before commit is created	Linting, formatting, tests
<code>prepare-commit-msg</code>	Before message editor opens	Add ticket number automatically
<code>commit-msg</code>	After message is entered	Validate Conventional Commits format
<code>post-commit</code>	After commit is created	Notifications, CI trigger
<code>pre-push</code>	Before <code>git push</code>	Run test suite before pushing

Example: `pre-commit` Hook

```
#!/bin/sh
# .git/hooks/pre-commit
# Run ESLint before every commit

echo "Running ESLint..."
npx eslint --ext .js,.ts src/
if [ $? -ne 0 ]; then
    echo "ESLint failed. Fix errors before committing."
    exit 1
fi
echo "ESLint passed."
```

```
# Make it executable
chmod +x .git/hooks/pre-commit
```

Sharing Hooks with Your Team

`.git/hooks/` is not committed to the repository. To share hooks:

```
# Option 1: Store hooks in a versioned directory
mkdir .githooks
cp .git/hooks/pre-commit .githooks/pre-commit

# Configure Git to use the custom hooks directory
git config core.hooksPath .githooks

# Commit .githooks/ to the repository
git add .githooks && git commit -m "chore: add shared git hooks"
```

Popular hook management tools

- **Husky** (Node.js projects) — `npm install husky --save-dev`
- **pre-commit** (Python) — declarative YAML hook configuration

Server-Side Hooks

Run on the remote server (GitHub, GitLab, Bitbucket use webhook equivalents):

Hook	When it runs	Common use
pre-receive	Before any refs are updated	Enforce policies, reject invalid commits
update	Per branch being updated	Branch-specific rules
post-receive	After all refs are updated	Deploy, notify CI/CD

On platforms like GitHub, these are replaced by **branch protection rules**, **Actions workflows**, and **webhooks**.

Part 5 — Plumbing vs. Porcelain

Porcelain Commands

High-level, user-friendly commands built for humans:

```
git add      git commit  git push     git pull
git checkout git merge  git rebase   git log
git status   git diff    git clone    git fetch
```

These are the commands you use every day. They combine multiple low-level operations.

Plumbing Commands

Low-level commands that operate directly on Git objects:

```
git hash-object      # compute and store an object hash
git cat-file         # read an object from the database
git ls-files         # list tracked files (the index)
git update-index     # manipulate the index directly
git write-tree       # create a tree object from the index
git commit-tree      # create a commit object
git read-tree        # read a tree into the index
git update-ref       # update a branch pointer
git rev-parse        # resolve a ref/alias to a SHA
git for-each-ref     # iterate over all refs
```

Plumbing in Practice

```
# Resolve HEAD to a full SHA
git rev-parse HEAD
# a3f7c218b9e4c1d2f3a4b5c6d7e8f9a0b1c2d3e4

# List all branches with their commit SHAs
git for-each-ref --format='%(%(refname:short) %(objectname:short))' refs/heads/

# See exactly what is staged (the index)
git ls-files --stage

# Count commits between two refs
git rev-list --count main..feature/auth
```

Plumbing commands are the building blocks used by porcelain commands — and by Git hosting platforms, CI tools, and scripts.

Summary — Git Internals

Concept	Key point
Blob	Raw file content, addressed by SHA
Tree	Directory snapshot mapping names to blobs/trees
Commit	Snapshot pointer + parent + metadata
<code>.git/objects/</code>	Content-addressable store for all Git objects
<code>.git/refs/</code>	Branch and tag pointers (human-readable names for SHAs)
SHA hashing	Immutable, self-verifying, deduplicated storage
Hooks	Scripts in <code>.git/hooks/</code> triggered by Git events
Plumbing	Low-level Git primitives for scripting and automation